Desarrollo de una aplicación con OCR y LLM

Jorge Casco Seco

jorgecs@unex.es

30 Abril 2026

Descripción del proyecto

Se debe realizar una aplicación web completa que funcione de forma similar a Google Lens pero permitiendo hacer preguntas sobre tu propia imagen:

- Subida de una imagen (png, jpg…) que contenga texto escrito, ya sea manuscrito o a máquina.
- Escanear la imagen utilizando una herramienta de OCR para obtener el texto.
- Utilizar este texto como contexto para un LLM.
- Permitir la realización de preguntas sobre este texto.

Objetivos

Hasta ahora se han desarrollado prototipos simples que permiten hacer uso de LLM y visión artificial por separado, pero en proyectos reales, estas tecnologías se utilizan en conjunto para obtener un flujo completo.

Esta actividad está orientada a evaluar de forma parcial la consecución de los siguientes resultados de aprendizaje del módulo de Programación de Inteligencia Artificial:

- **RA3. Evalúa las mejoras en los negocios integrando convergencia tecnológica.**
- **RA4. Evalúa modelos de automatización industrial y de negocio relacionándolos con los resultados esperados por las empresas.**

Se emplearán los criterios de evaluación asociados a ellos, tal y como se recogen en el Real Decreto 279/2021.

Desarrollo

A continuación se describe el proceso recomendado para implementar la aplicación paso a paso.

1. Preparación del entorno
   - Crear y activar un entorno virtual de Python.
   - Instalar dependencias (`fastapi`, `fastapi[standard]`, `transformers`, `google-genai`, etc.).
   - Crear un fichero `.env` con la clave `GEMINI_API_KEY`.
   - Verificar que la aplicación arranca y muestra el frontend estático.
2. Estructura base del backend en FastAPI
   - Definir una aplicación FastAPI y montar la carpeta `static`.
   - Crear un endpoint raíz `GET /` que devuelva `static/index.html`.
   - Definir modelos de datos con Pydantic para validar las peticiones y respuestas.
3. Endpoint OCR (`POST /api/upload`)
   - Recibir una imagen desde el frontend.
   - Validar el tipo de archivo (por ejemplo, `image/png`).
   - Convertir la imagen a formato adecuado para OCR.
   - Ejecutar el OCR para extraer texto.
   - Devolver al frontend el contexto completo.
4. Gestión del contexto en frontend
   - Al recibir la respuesta OCR, mostrar al usuario una vista previa del texto.
   - Guardar el `contexto` para enviarlo después en cada pregunta de chat.
   - La tecnología frontend a utilizar es de libre elección.
5. Endpoint de chat (`POST /api/ask`) en modo stateless

- Recibir `pregunta`, `contexto` e `historial`.
- Validar que el contexto no esté vacío.
- Construir un prompt aumentado con reglas de respuesta, contexto, historial y pregunta actual.
- Enviar ese prompt a la estrategia de consulta con fallback.

6. Estrategia de fallback de modelos LLM

- Intentar primero el modelo principal de Gemini.
- Si falla tras los reintentos definidos, intentar el modelo fallback de Gemini.
- Si también falla, usar un modelo local de Hugging Face.
- Si todos fallan, devolver un mensaje de error controlado para el usuario.

7. Comprobar integración frontend-backend

- Flujo completo:
  - Subir imagen -> OCR -> mostrar contexto.
  - Enviar pregunta con contexto e historial -> recibir respuesta.
  - Mostrar la respuesta en el chat.
- Mantener historial y contexto en frontend para reenviarlo en cada petición (enfoque stateless en backend).

8. Pruebas y documentación

- Funcionamiento correcto con las imágenes de prueba (recuperación correcta del contexto y chat coherente).
- Documentación de decisiones de diseño (modelo OCR), limitaciones detectadas (calidad OCR, tipos de imagen, tiempos de respuesta) y cómo arrancar la aplicación (por si se usa una tecnología diferente en el frontend a lo visto en clase). Se valorará el formato y la limpieza.

Por tanto, el flujo que debe ver el usuario una vez finalizada la aplicación es el siguiente:

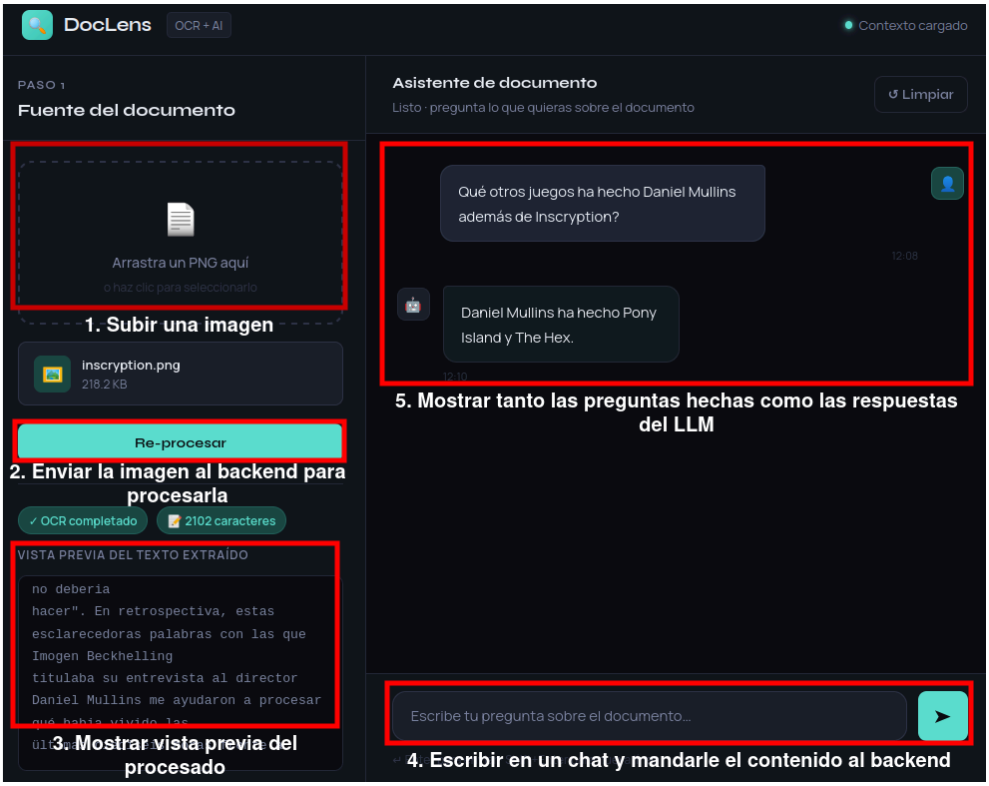

Figura 1: Flujo de la aplicación

Por si existiesen dudas, a continuación se muestra un diagrama de secuencia con los pasos que se deben realizar de forma interna, comunicando frontend y backend:

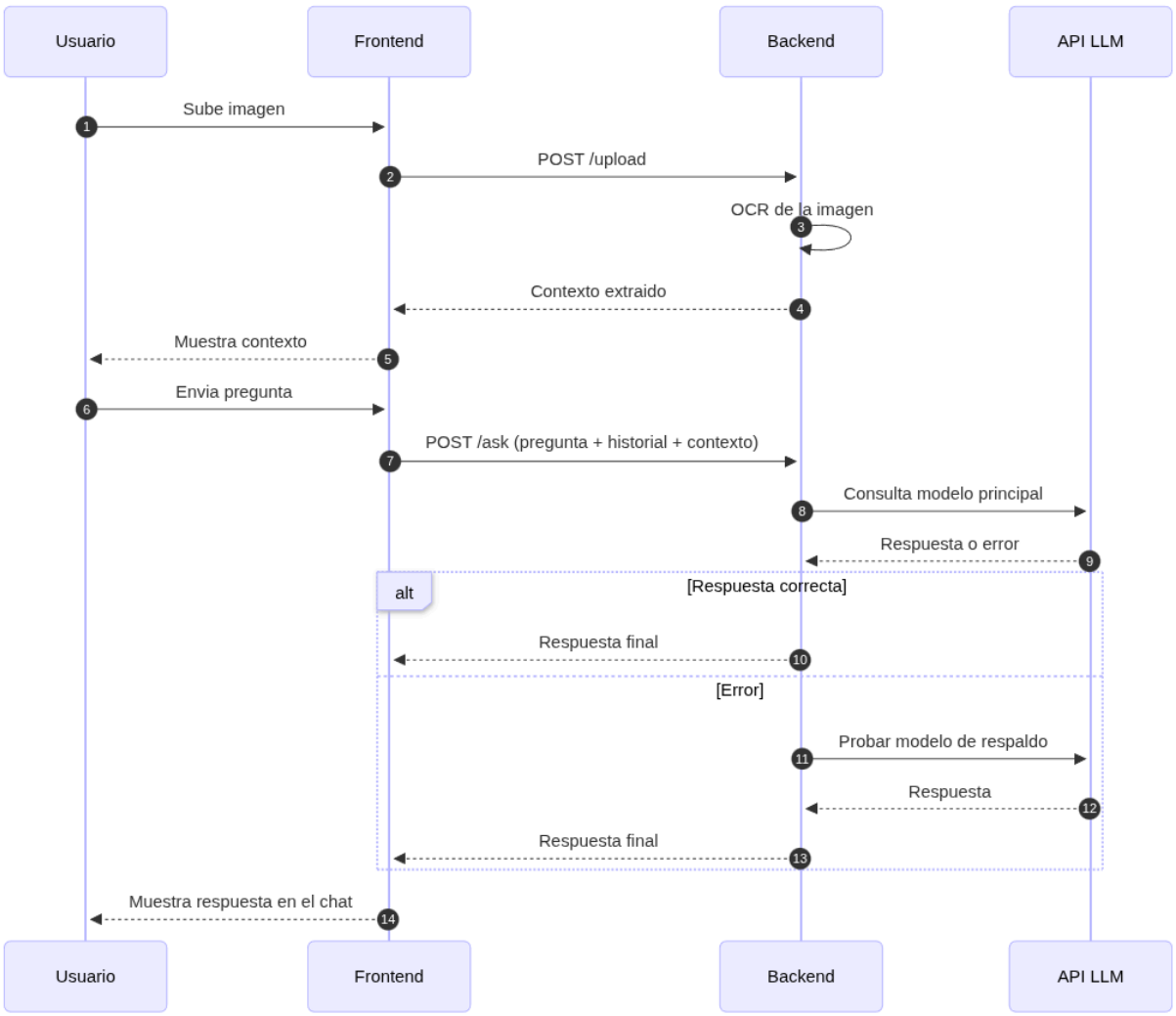

Figura 2: Diagrama de secuencia de la aplicación

En el diagrama, API LLM representa la estrategia de fallback:

- Primero se intenta con Gemini principal
- Si falla, se prueba un segundo modelo de Gemini
- Si vuelve a fallar, se utiliza un modelo local de Hugging Face
- Si ninguno responde correctamente, se devuelve un mensaje de error al usuario.

Entrega

- Fecha límite: **10 Mayo 2026**
- Fichero **ZIP** que incluya:
  - **main.py**: Backend en FastAPI.
  - **requirements.txt**: Versiones de las librerías utilizadas (pip freeze).
  - **index.html** u otra tecnología de frontend.
  - **Imágenes** utilizadas para las pruebas.
  - **Documentación** explicando decisiones tomadas (tecnología OCR, problemas durante el desarrollo, imágenes que funcionan y que no funcionan…) y cómo ejecutar la aplicación.

Calificación

Criterio	Peso
Flujo, calidad y usabilidad del frontend	10%
Estructura del backend con FastAPI	15%
Endpoint `/api/upload` y OCR funcional	15%
Complejidad técnica OCR (Tesseract o Paddle)	10%
Endpoint `/api/ask` y uso de Gemini	15%
Fallback de modelo	15%
Gestión stateless (contexto e historial)	15%
Integración y documentación	5%

Calificación máxima total:
10 puntos